

结构感知单核融合的机械臂批量逆运动学 CUDA 加速方法

刘霄鹏

武汉科技大学机械工程学院, 武汉 430081

文章编号: (预留) DOI: (预留)

摘要: 针对固定六自由度机械臂中小批量逆运动学 (inverse kinematics, IK) 求解中单种子收敛稳定性不足、通用多阶段图形处理器 (graphics processing unit, GPU) 流水线固定开销较高的问题, 提出一种结构感知单核融合的计算统一设备架构 (compute unified device architecture, CUDA) 加速方法。该方法首先将运动学参数、解析雅可比和 Sobol 多起点迭代编码为固定结构; 然后采用目标块映射在块内并行评估候选解; 最终融合正运动学、雅可比组装、正规方程求解和最佳候选输出。实验结果表明, 该方法在可达目标批量求解中能够兼顾一次性成功率和吞吐量。近奇异、近限位和轨迹实验进一步说明其适合作为规划前端候选生成器。

关键词: 逆运动学; 批量并行计算; 统一计算设备架构 (CUDA); Levenberg-Marquardt 算法; 核函数融合

中图分类号: TP391.4 **文献标志码:** A

收稿日期: (预留); **修回日期:** (预留)。

Structure-Aware Single-Kernel Fusion CUDA Acceleration for Batch Inverse Kinematics of Robotic Manipulators

LIU Xiaopeng

School of Mechanical Engineering, Wuhan University of Science and Technology, Wuhan 430081, China

Abstract: Aiming at the insufficient convergence stability of single-seed inverse kinematics (IK) and the fixed overhead of generic multi-stage graphics processing unit (GPU) pipelines for medium-scale batches of fixed six-degree-of-freedom manipulators, a structure-aware single-kernel fusion acceleration method based on compute unified device architecture (CUDA) is proposed. The method first encodes kinematic parameters, analytical Jacobian evaluation, and Sobol multi-start iterations as a fixed structure. Then, a target-block mapping evaluates candidate solutions in parallel inside each block. Finally, forward kinematics, Jacobian assembly, normal-equation solution, and best-solution output are fused. Experiments show that the method balances one-shot success rate and throughput for reachable batch targets. Near-singular, near-limit, and trajectory experiments further indicate that it is suitable as a candidate generator for planning front-ends.

Keywords: inverse kinematics; batch parallel computing; compute unified device architecture (CUDA); Levenberg-Marquardt algorithm; kernel fusion

0 引言

但子问题数量足够多, 可用并行性充足。

机械臂批量逆运动学 (inverse kinematics, IK) 是采样式运动规划、轨迹优化和机器人学习中的核心计算环节 [1, 2, 3, 4]。近年来, 向量化采样式规划进一步表明, 正运动学、碰撞检测和局部 IK 等细粒度子程序的批量并行化对实时规划前端具有直接价值 [5]。对 6 自由度串联机械臂而言, 单个 IK 问题规模极小: 6×6 雅可比矩阵、6 维线性系统、每迭代约 1 650 次双精度浮点运算。然而, 当批量规模 N 达到数百至数千时, 计算范式发生根本转变——问题从“单个 6 维非线性最小二乘”变为“ N 个独立 6×6 小矩阵运算的批量并行调度问题”。在这一范式下, GPU 的固定调度开销 (kernel launch $\approx 5\text{--}10 \mu\text{s}/\text{次}$) 和主机-设备同步延迟可能主导实际性能 [6, 7], 形成小矩阵批量计算的结构性矛盾: 每个子问题计算量极小, 不足以摊薄调度开销;

这一矛盾并非 IK 特有, 而是广泛存在于机器人学中任何涉及固定小规模线性代数批量计算的场景。现有 GPU 库对小矩阵支持不足: NVIDIA cuBLAS 批量 GEMM 接口 (cublasGemmBatchedEx) 设计目标为 $m, n, k \geq 16$, 对 6×6 规模存在 $> 10 \mu\text{s}/\text{次}$ 的固定调用开销; cuSOLVER 批量 LU/QR 分解同理 [6]。小矩阵批处理领域已有研究表明, 寄存器分块、共享内存常驻和批量调度策略对 tiny matrices 的性能影响显著 [22]。传统迭代 IK 可追溯到 resolved-rate control、阻尼最小二乘和误差阻尼伪逆等方法 [12, 13, 14, 15, 16], 这些方法在单目标场景中易于部署, 但当目标数达到百级以上时, 每目标毫秒级耗时难以满足实时规划前端需求。在 GPU 加速领域, cuRobo[8] 采用 CUDA Graph 和粒子群-L-BFGS 优化器实现大规模并行 IK, 后续工作进一步从可变精度、混合采样优化和工业扩展自由度

应用等角度推进 GPU 运动生成 [9, 10, 11]。不过, 通用多阶段优化结构在 $N \leq 1000$ 的中小批量场景中仍存在固定调度成本。本文关注的问题是: 针对 6×6 这一固定且极小的矩阵规模, 将运动学结构信息编码为编译期常量并融合为单核函数后, 能否在中等精度阈值下获得稳定吞吐收益?

本研究提出运动学结构驱动的 CUDA 小矩阵加速方法, 简称 OPT4C。其核心不是将已有算法库移植至 GPU, 而是将运动学结构信息 (关节轴方向、解析几何关系、固定 Sobol 种子数 $K = 16$) 作为编译期常量编码进核函数, 使 FK、雅可比组装、LM 迭代更新与块内候选筛选全部在单一 kernel 内完成。该研究在 NVIDIA GeForce RTX 4060 Laptop GPU 上对方法进行了系统实验验证和微架构分析。

该方法的应用场景界定为: 批量目标数 N 在数百量级 ($N \leq 1000$)、机械臂构型固定为 6-DOF、碰撞检测不在 IK kernel 内处理的实时规划前端。对于通用多阶段 GPU pipeline, 小批量 IK 中 kernel launch、host-device 同步和全局内存中转会显著影响端到端延迟; 本文通过单 kernel fusion 压缩上述固定调度成本。fusion 之后, 主要瓶颈转移为 FP64 小矩阵求解、三角函数和标量寄存器运算的指令延迟。

1 问题定义与评价协议

1.1 批量 IK 问题形式化

令 $q \in \mathbb{R}^6$ 为 UR10 六个旋转关节角, 正运动学 $T_{ee}(q) \in \text{SE}(3)$ 将关节角映射至末端 tool0 位姿。对目标 T^* , 定义平移误差 $e_p(q) = p_{ee}(q) - p^* \in \mathbb{R}^3$ 和旋转误差 $e_R(q) = \log(R^{*\top} R_{ee}(q))^\vee \in \mathbb{R}^3$ 。单目标 IK:

$$\min_{q \in [q_{\min}, q_{\max}]} \frac{1}{2} \|e_p(q)\|^2 + \frac{\alpha_R}{2} \|e_R(q)\|^2. \quad (1)$$

批量 IK 需对 N 个独立目标并行求解式 (1), 输出每目标的最佳候选解。值得注意的是, 6-DOF 串联机械臂在关节限位和奇异位形附近存在多解性和可解性退化问题 [15], 多种子策略是应对该挑战的有效手段。

1.2 统一评价协议

- Strict 成功率: 同时满足 $e_p < 5 \text{ mm}$ 且 $e_R < 1^\circ$ 的目标比例;
- 全样本位置误差 p_{95} : 全部 N 个目标 (含失败) 位置误差的 95 分位数;
- 原始吞吐量: $N/\bar{t}_{\text{gpu}}$, $\bar{t}_{\text{gpu}}$ 为 CUDA event 测量的 GPU 流时间均值 (30 次重复)。

2 运动学结构驱动的 CUDA 小矩阵加速方法

本章按自底向上的顺序展开: 首先将运动学参数编码为 GPU 常量内存, 然后设计融合 FK 函数实现解析雅可比的高效组装, 接着给出低控制流复杂度 LM 迭代算法, 最后将所有组件融合为单一 target-block 核函数。

2.1 运动学模型至 GPU 常量内存的编译期编码

UR10 运动学参数来源于官方 URDF 模型 [17]。将以下编译期确定的信息编码为 CUDA `__constant__` 内存:

- 6 个关节轴单位向量 $a_i \in \mathbb{R}^3$ ($a_0 = Z, a_1 = Y, a_2 = Y, a_3 = Y, a_4 = -Z, a_5 = Y$);
- 6 个连杆偏移矩阵 $\text{origin}_i \in \text{SE}(3)$;
- 腕 3 至 tool0 固定变换 $T_{\text{wrist3_to_tool0}}$;
- 关节限位 $[q_{i,\min}, q_{i,\max}]$ 和障碍函数参数。

Ada Lovelace 架构 (SM 8.9) 提供 48 KB 常量缓存, 所有线程块可通过广播总线低延迟访问。这些参数总计约 2 KB——不足缓存容量的 5%, 可避免明显的常量缓存逐出。

2.2 融合 FK 函数与解析雅可比组装

数值差分雅可比需对每个关节执行正负扰动 FK, 每迭代 $6 \times 2 = 12$ 次额外 FK 调用。对平均 20 次 LM 迭代, 累计 ≈ 260 次 FK 调用——FK 成为计算瓶颈。本研究设计的融合 FK 函数在单次前向传播中同步输出三个层次的几何量: 末端 tool0 位姿 T_{ee} 、各关节世界位置 p_i 、各关节世界转轴 $z_i = R_{3 \times 3}^{(i)} \cdot a_i$ 。算法 1 给出了融合 FK 的计算流程。

获得 (p_i, z_i) 后, 6×6 解析雅可比矩阵的组装仅需 6 次叉积运算 [1, 18]:

$$J_{v,i} = z_i \times (p_{ee} - p_i), \quad J_{\omega,i} = z_i, \quad i = 0, \dots, 5. \quad (2)$$

与有限差分方案相比, 解析组装的优势主要体现在三个方面: (1) 减少 FK 调用次数, 以 6 次叉积替代 12 次扰动 FK; (2) 避免差分步长调参; (3) 在奇异附近或尺度不一致时提供更稳定的梯度方向。本文不将解析雅可比的收益归因于差分截断误差接近 5 mm 阈值, 而是将其定位为降低计算量和提高 LM 方向稳定性的结构化实现。

表 1 算法 1 融合 FK 函数 (含坐标系输出)

```

// 输入: 6 维关节角向量
// 输出: 末端 tool0 位姿、关节世界位置、关节转轴
Input:  $q[0:5]$ 
Output:  $T_{ee}, p[0:5][0:2], z[0:5][0:2]$ 
// 初始化 4×4 齐次变换矩阵为单位阵
 $T \leftarrow I_4$ 
for  $i = 0$  to 5 do
// 施加连杆偏移矩阵, 记录关节世界位置
 $T \leftarrow T \cdot \text{origin}[i]$ 
 $p[i] \leftarrow [T_3, T_7, T_{11}]$ 
// 提取当前累积旋转子块, 计算世界转轴向量
 $R_{3 \times 3} \leftarrow T_{0:2,0:2}$ 
 $z[i] \leftarrow R_{3 \times 3} \cdot \text{axis}[i]$ 
// 施加绕关节轴  $i$  的旋转
 $T \leftarrow T \cdot \text{Rodrigues}(\text{axis}[i], q[i])$ 
// 施加腕 3 至 tool0 固定变换
 $T_{ee} \leftarrow T \cdot T_{\text{wrist3\_to\_tool0}}$ 
return  $T_{ee}, p, z$ 

```

2.3 低控制流复杂度 LM 迭代算法

每个 Sobol 种子独立执行最多 60 次 LM 迭代 [19, 20]。加权误差 $e = [e_p^\top, \alpha_R e_R^\top]^\top \in \mathbb{R}^6$ 与解析雅可比 $J \in \mathbb{R}^{6 \times 6}$ 构造正规方程:

$$(J^\top J + \lambda I) \Delta q = -(J^\top e + w_{\text{limit}} \nabla \Phi_{\text{limit}}), \quad (3)$$

其中 $\Phi_{\text{limit}} = \sum_{j=0}^5 \phi(q_j)$ 为关节限位二次障碍函数, $\phi(q_j) = \max(0, m - (q_j - q_{j,\min}))^2 + \max(0, m - (q_{j,\max} - q_j))^2$, margin $m = 0.087 \text{ rad} (\approx 5^\circ)$, 默认 $w_{\text{limit}} = 0.03$ 。关键设计: 阻尼因子 λ 摒弃传统 LM 的 acceptance-rejection 回滚分支——若 $\rho > 0$ 则 $\lambda \leftarrow 0.5\lambda$, 否则 $\lambda \leftarrow 2\lambda$, $\lambda \in [10^{-6}, 0.5]$ ——新候选解总是被接受。该设计降低了控制流复杂度和状态保存开销, 但不同 seed 的阻尼更新和收敛判定仍可能导致 warp 内分支分化, 因此本文仅将其称为低控制流复杂度实现。6×6 正规方程的求解采用寄存器级高斯消元 (Gaussian elimination with partial pivoting) 在 6 次主元消去中完成, 所有中间变量保持在寄存器中。

算法 2 给出了单个种子的完整 LM 迭代流程。

2.4 Target-Block 融合映射: 单 Kernel 端到端执行

上述 LM 迭代对 N 个目标 $\times K$ 个种子独立执行, 共 $N \times K$ 个独立求解任务。朴素逐目标/逐种子实现的启动次数可能随 N 和 K 增长, 而更常见的 batched GPU pipeline 启动次数主要随优化阶段数和迭代阶段数增长。本文的优势不是相对一个极差的逐

表 2 算法 2 单种子 LM 迭代 (低控制流复杂度, 寄存器级小矩阵求解)

```

// 输入: Sobol 种子、目标位姿、最大迭代次数、初始阻尼
// 输出: 最优关节角、代价、成功等级
Input:  $q_{\text{seed}}, T_{\text{target}}, \text{max\_iter}=60, \lambda_0=0.01$ 
Output:  $q_{\text{best}}, \text{cost\_best}, \text{success\_rank}$ 
// 初始化
 $q \leftarrow q_{\text{seed}}; \lambda \leftarrow \lambda_0; \text{cost\_best} \leftarrow \infty$ 
for  $\text{iter} = 1$  to  $\text{max\_iter}$  do
// 调用算法 1: 融合 FK 输出末端位姿 + 关节位置 + 转轴
 $(T_{ee}, p, z) \leftarrow \text{FusedFK}(q)$ 
 $e_p \leftarrow p_{ee} - p_{\text{target}}; e_R \leftarrow \log(R^\top R_{ee})$ 
// 解析雅可比组装: 6 次又积替换 12 次扰动 FK
for  $i = 0$  to 5 do
 $J_{0:2,i} \leftarrow z[i] \times (p_{ee} - p[i])$ 
 $J_{3:5,i} \leftarrow z[i]$ 
// 正规方程 + 寄存器级 6×6 高斯消元
 $H \leftarrow J^\top J + \lambda I; g \leftarrow J^\top e$ 
 $\Delta q \leftarrow \text{GaussElim6} \times 6(H, -g - w \nabla \Phi)$ 
 $q_{\text{trial}} \leftarrow q + \Delta q; \text{clamp to } [q_{\min}, q_{\max}]$ 
// 损失评估: 位姿代价 + 限位障碍
 $\text{cost} \leftarrow 0.5 \|e\|^2 + w_{\text{limit}} \Phi(q_{\text{trial}})$ 
 $\rho \leftarrow (\text{cost\_old} - \text{cost}) / \text{cost\_old}$ 
// 阻尼自适应: 总是接受 trial, 仅调节  $\lambda$ 
if  $\rho > 0$  then  $\lambda \leftarrow 0.5\lambda$  else  $\lambda \leftarrow 2\lambda$ 
 $q \leftarrow q_{\text{trial}}$ 
if  $\text{cost} < \text{cost\_best}$  then  $(q_{\text{best}}, \text{cost\_best}) \leftarrow (q, \text{cost})$ 
// 收敛判定
if converged( $\|e_p\| < 5\text{mm}, \|e_R\| < 1^\circ$ ) then break
return  $(q_{\text{best}}, \text{cost\_best}, \text{success\_rank})$ 

```

目标 baseline，而是将 FK、Jacobian、LM update 和候选选择融合进单个 target-block kernel，减少阶段间全局内存中转和 host-device 同步。

表 3 算法 3 Target-Block 融合核函数（单 Kernel，共享内存块内选择）

```
// 核函数签名与启动配置
Kernel: IK_LM_Multiseed_Block_Target
Grid: N blocks, Block: 32 threads
Constants: T_targets[N], seeds[N][K][6] (global)
Shared: s_cand[K][stride] (K=16, stride=16)
Output: best[N][kBestStride]

// 每个 Block 处理一个目标；16 个 lane 各执行一个 Sobol 种子
tid ← blockIdx.x; lane ← threadIdx.x
if lane < K then
// 加载种子，调用算法 2 执行 LM 迭代
qseed ← seeds[tid][lane]
(qbest, cost, rank) ← LM_Iter(qseed, T_targets[tid])
// 候选解写入共享内存（仅用于块内候选缓存）
for j = 0 to 5 do s_cand[lane][j] ← qbest[j]
s_cand[lane][6..8] ← 目标平移分量
s_cand[lane][9] ← cost
s_cand[lane][10] ← rank
s_cand[lane][11] ← (near_limit ? 1 : 0)
// 同步全部 16 lane 的候选解写入
__syncthreads()
// 仅 lane 0 执行块内三级层次化选择
if lane = 0 then
best_idx ← 0
for k = 1 to K - 1 do
// 优先级 1: success rank
if s_cand[k][10] > s_cand[best_idx][10] then
best_idx ← k
else if equal then
// 优先级 2: near_limit (0 优于 1)
if s_cand[k][11] < s_cand[best_idx][11] then
best_idx ← k
// 优先级 3: pose cost 升序
else if equal ∧ s_cand[k][9] < s_cand[best_idx][9]
then best_idx ← k
// 最佳候选从共享内存写回全局内存
for j = 0 to 5 do best[tid][j] ← s_cand[best_idx][j]
best[tid][6] ← s_cand[best_idx][9] (pose cost)
best[tid][7] ← s_cand[best_idx][10] (success rank)
```

算法 3 的三个关键数据结构设计为：

(1) **Grid-Block 映射**。<<<N,32>>> 启动配置将 N 个目标位姿一对一映射至 N 个线程块，每块 32 线程中 lane 0–15 各承载一个 Sobol 种子 ($K = 16$)，lane 16–31 空闲。这种映射使每个目标的计算完全局部化于一个 SM 内——无跨块通信、无全局同步。

(2) **共享内存候选缓存**。候选解数组 $s_cand[16][16]$ 仅用于块内 16 个候选解的缓存

和最终选择，总容量约 2 KB。候选写入和读取只发生在每个 block 内，访问次数相对 LM 主循环较少；结合 CUDA 共享内存与访存优化建议 [23]，Nsight Compute 采样显示共享内存访问不是主要瓶颈。本文后续不将共享内存访存冲突作为主要优化目标，而将重点放在 FP64 小矩阵求解、三角函数和标量寄存器运算上。

(3) **块内三级层次化选择**。lane 0 扫描 16 个候选解的优先级：success rank (Strict > Medium > Loose > Fail) → near_limit 标志位 (0 优于 1) → pose cost 升序。该选择逻辑在共享内存和寄存器中完成，不引入额外 kernel launch。

图 1–2 展示了线程映射架构和完整算法流程。

2.5 Kernel Fusion 的调度开销分析

表 4 给出了不同实现路径的阶段开销对比。对于本文方法，核心求解阶段由单个 target-block kernel 完成，H2D、kernel、D2H 之外无中间 host-device 同步。该结构差异在 $N \leq 1000$ 的低批量场景中较为重要；cuRobo 虽通过 CUDA Graph 缓解部分启动延迟，但其粒子初始化、评估和 L-BFGS 更新仍属于多阶段优化流水线。

表 4 Kernel Fusion 前后的调度开销对比

实现方式	Kernel 阶段	全局内存往返	Host 同步
朴素逐目标实现	$O(NK)$	多次	多次
多阶段 batched pipeline	$O(\text{stage} \times \text{iter})$	多次	少量
本文 OPT4C	1 core kernel	最少	无中间同步

3 实验设置

所有实验在单块 NVIDIA GeForce RTX 4060 Laptop GPU 上进行 (Ada Lovelace, SM 8.9, 24 组 SM, 8 GB GDDR6, FP64 理论峰值 ≈ 0.18 TFLOPS)。软件栈为 CUDA Toolkit 13.3 (nvcc V13.3.33)、驱动 610.43.02、GCC 11.4.0，编译选项为 -O3 -DCMAKE_BUILD_TYPE=Release。本文方法的求解器参数为 variant = opt4c_block_target、precision = fp64、limit_gradient = analytic、 $K = 16$ (Sobol 序列 [21])、预热 10 次、重复测量 30 次。所有 GPU 时间由 CUDA event API 测量 (cudaEventElapsedTime)，排除首轮预热和内存分配时间。

目标位姿采用“随机关节角 → FK”策略生

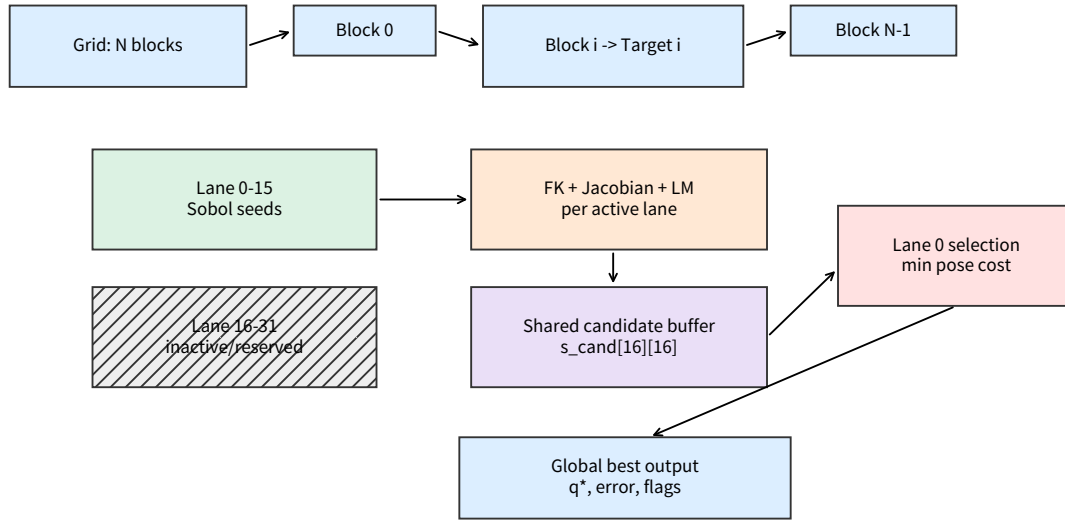


图 1 Target-Block 线程映射架构。一个 block 处理一个目标位姿，lane 0–15 分别处理 16 个 Sobol 起点，lane 16–31 保留为空闲线程；候选解暂存于共享内存，并由 lane 0 完成块内最佳候选选择后写入全局输出。

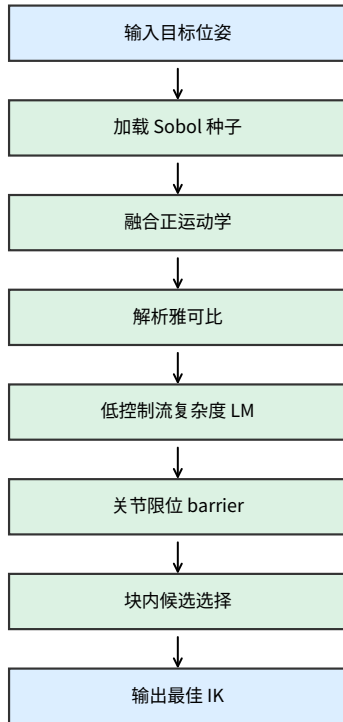


图 2 求解器完整算法流程

成，确保物理可达。固定随机种子 42，生成 $N = 100, 200, \dots, 1000$ （步长 100，共 10 个规模）的目标位姿 raw 文件（ $[N, 16]$ double，行优先 4×4 齐次矩阵）。cuRobo 对比采用 cuRobo-Graph 模式，碰撞检测关闭，CUDA Graph 开启，并分别报告默认单种子配置（cuRobo-Graph-K1）和同等 16 种子配置（cuRobo-Graph-K16）。cuRobo 的 graph capture、solver 构造和内存分配不计入单次 GPU solve 时间；预热与重复测量阶段计入 CUDA Graph replay 后的求解时间。所有方法输出的关节角均经同一 UR10 URDF 外部 FK 管线复评。本文以 Strict 作为主指标，并用 Loose、Medium 和 Ultra 作为辅助阈值：Loose 为 $30 \text{ mm} / 10^\circ$ ，Medium 为 $10 \text{ mm} / 5^\circ$ ，Strict 为 $5 \text{ mm} / 1^\circ$ ，Ultra 为 $2 \text{ mm} / 0.5^\circ$ ；四级阈值均基于复评误差计算。

4 实验结果与分析

4.1 静态批量 IK 综合性能

表 5 给出 10 个规模的综合结果。全部规模通过单调性检查（Loose SR $\geq$ Medium SR $\geq$ Strict SR），零 NaN/Inf 异常。Strict 成功率 0.940–0.960， $N \geq 500$ 时全样本位置误差 p_{95} 均低于 5 mm Strict 阈值（ $N = 300$ 时个别失败样本影响而升高 p_{95} ——小样本下单个失败对分位数影响更大）。平均迭代 19–21 次，近限位比例 $< 1.0\%$ 。

吞吐量增长模式。 $N = 100 \rightarrow 1000$ 吞吐量仅提

表 5 静态批量 IK 综合性能 ($N = 100 \sim 1000$, 步长 100)

N	GPU 时间/ms	吞吐量/(t · s ⁻¹)	Strict SR	p_{95} /mm	迭代次数	近限值
100	6.658	15019.0	0.960	4.38	21.0	0.010
200	13.724	14572.8	0.950	5.49	20.0	0.005
300	19.334	15516.5	0.940	27.04	20.0	0.007
400	24.090	16604.1	0.945	15.44	20.0	0.005
500	28.837	17338.9	0.954	4.34	20.0	0.004
600	33.677	17816.3	0.953	4.36	20.0	0.005
700	39.329	17798.8	0.953	4.78	19.5	0.006
800	44.421	18009.3	0.951	4.91	19.0	0.006
900	49.330	18244.6	0.951	4.90	19.0	0.007
1000	54.868	18225.5	0.954	4.56	19.0	0.007

升 21% ($1.50 \rightarrow 1.82 \times 10^4$), 符合典型的固定开销分摊曲线——该趋势可由 Amdahl 定律刻画: lane 0 串行候选选择 (≈ 0.005 ms/target) 构成不可并行的串行分量。GPU 流时间与 N 严格线性 ($R^2 > 0.999$, 斜率 0.0549 ms/target), 不随批量增大而退化——这是 target-block 映射结构确定性的直接验证。

GPU 时间组成。如图 3 所示, kernel fusion 后 OPT4C 的总时间主要由核心求解 kernel 贡献, H2D、D2H 和 launch 时间均处于次要地位。这说明本文方法的后续优化重点不应继续放在减少调度次数, 而应转向降低单迭代 FP64 小矩阵求解、三角函数和标量寄存器运算的代价。

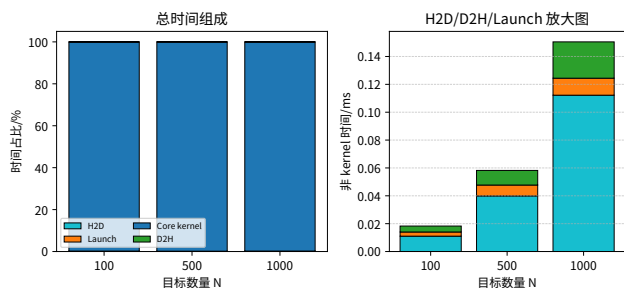


图 3 OPT4C 时间组成

为从系统时间线层面验证单 kernel 执行路径, 使用 Nsight Systems 对 $N = 1000, K = 16$ 配置进行采样, 命令包含 10 次 warmup 和 30 次 repeat。采样结果如图 4 所示, 40 次重复均调用同一个核心 IK kernel, kernel 平均时长为 57.96 ms; H2D/D2H 拷贝总时长约 5.24 ms, memset 总时长约 0.025 ms。该结果说明每次 batch 求解的核心计算阶段未出现多阶段 kernel 往返, 支持本文关于 kernel fusion 执行路径的系统级判断。采样时本机不允许 CPU context switch tracing, 因此本文仅使用 CUDA kernel、memcpy 和 memset 事件进行

分析。

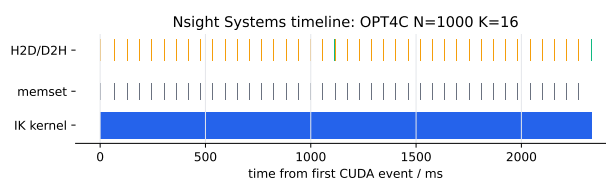
图 4 Nsight Systems 时间线 ($N = 1000, K = 16$)

图 5-6 给出了吞吐量与成功率的密集对比, 全样本位置误差 p_{95} 的数值对比见上文。图 5-6 为默认配置对比, 即 OPT4C-K16 与 cuRobo-Graph-K1; 同等种子数 $K = 16$ 的对比见表 6 和图 7。

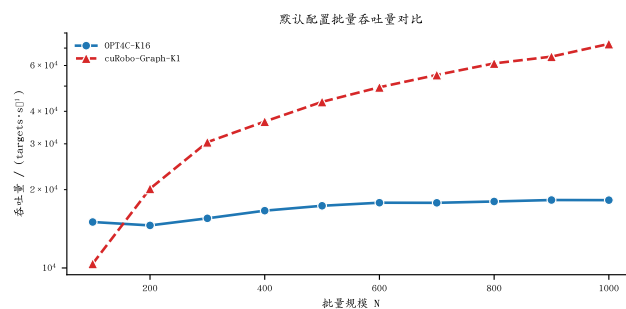


图 5 默认配置批量吞吐量对比 (OPT4C-K16 与 cuRobo-Graph-K1)

全样本位置误差 p_{95} 对比如下: 本方法 p_{95} 在 $N \geq 500$ 时低于或接近 5 mm Strict 阈值 (4.34–4.91 mm); cuRobo $K=1$ 的 p_{95} 为 74–115 mm, 被失败样本尾部影响而升高; cuRobo $K=16$ 降至 0.5–0.9 mm。5 mm/1° 阈值可作为采样式规划前端或中等精度任务中的评价阈值; 对于精密装配、插接和接触丰富任务, 该阈值仍不足, 需要更高精度的局部优化或视觉伺服闭环。

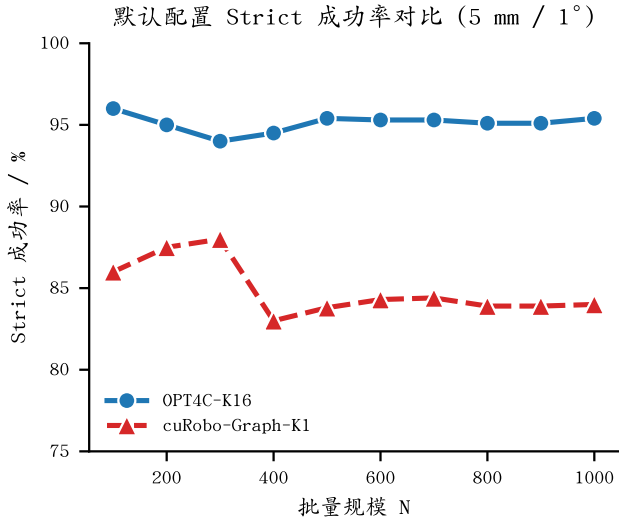


图6 默认配置 Strict 成功率对比 (OPT4C-K16 与 cuRobo-Graph-K1)

4.2 与 cuRobo 的系统级对比与公平性分析

表6给出了 $N = 100, 500, 1000$ 下四种配置的系统级对比: 本方法 $K = 16$ (默认)、本方法 $K = 1$ (消融)、cuRobo-Graph-K16 (同等种子数) 和 cuRobo-Graph-K1 (默认单种子)。该表同时列出全样本 $p95$ 、成功样本 $p95$ 和有效吞吐量, 用于区分“求解速度”和“满足 Strict 阈值的有效求解速度”。

默认配置对比。 $N = 100$ 时本方法在吞吐量和精度上同时领先 cuRobo $K=1$ (吞吐比 1.45:1, SR 0.960 vs 0.860)。 $N \geq 200$ 时 cuRobo $K=1$ 凭借粒子群并行搜索反超, $N = 1000$ 时吞吐达 7.26×10^4 targets/s, 但 Strict SR 仅 0.840, $p95$ 为 88.7 mm——约 16% 的失败样本将全样本误差影响而升高了两个数量级。

同等 16 种子对比。 将 cuRobo 种子数提升至与本方法相同的 $K = 16$ 后, cuRobo 的 L-BFGS 优化器展现出更强的收敛能力: Strict SR 达 0.980–0.988 (本方法 0.954–0.960), $p95$ 仅 0.5–0.9 mm (本方法 4.3–4.6 mm)。然而, 本方法的吞吐量是 cuRobo $K=16$ 的 1.67–1.79 倍 ($N = 1000$ 时 1.82×10^4 vs 1.04×10^4 targets/s)。该吞吐优势主要与单核函数融合减少阶段间全局内存中转和 host-device 同步有关, 同时也与本文固定结构 LM 的较轻量计算路径有关。由于 cuRobo-Graph-K16 追求更高成功率和更低残差, 其内部粒子初始化、评估与 L-BFGS 优化流程更复杂, 因此二者的吞吐差异不应仅归因于 kernel launch 或调度开销。

有效吞吐量分析。 若仅以 throughput $\times$ Strict SR 衡量, cuRobo-Graph-K1 在 $N = 1000$ 时仍具有更高有效吞吐; 但该指标未反映失败目标在规划管线中的重采样、回退或轨迹修补代价。cuRobo-Graph-K1 的

成功样本 $p95$ 仅为 0.28 mm, 说明其成功解精度很高; 同时, 全样本 $p95$ 被失败尾部抬高到 88.67 mm。对于要求单次 batch 输出高比例 Strict 解、且不希望引入额外回退逻辑的规划前端, OPT4C-K16 的一次性成功率更具工程可控性。

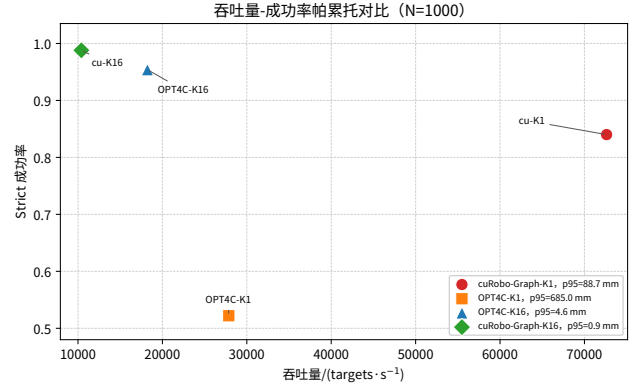


图7 吞吐量-成功率帕累托对比 ($N = 1000$)

注: 有效吞吐 = 吞吐量 $\times$ Strict SR, 反映实际成功求解的目标速率。cuRobo $K=1$ 全样本 $p95$ 被失败样本尾部影响而升高, 其成功样本 $p95$ 仅 0.1–0.3 mm。本方法 $p95$ 为 4.6 mm, 虽高于 cuRobo $K=16$ 的 0.9 mm, 但已满足本文 Strict 阈值 (< 5 mm), 适用于采样式规划前端等中等精度场景。

进一步地, 为避免单一 Strict 阈值掩盖质量差异, 本文在 $N = 1000$ 下对 Loose、Medium、Strict 和 Ultra 四级阈值进行扫描, 结果如表7和图8所示。四级阈值定义已在实验设置中给出, 其中 Strict 仍作为主评价指标, Ultra 仅用于观察更严格残差要求下的质量差异。cuRobo-Graph-K16 在 Ultra 阈值下仍保持 0.970 成功率, 说明其精度优势明显; OPT4C-K16 在 Strict 和 Medium 阈值下均为 0.954, 在 Loose 阈值下为 0.965, 表明其定位是满足 5 mm/1° 附近的工程阈值, 而不是追求亚毫米级精度。cuRobo-Graph-K1 的 Strict 成功率为 0.838, 但 Loose 成功率升至 0.896, 说明其失败样本中存在一部分中等误差解; OPT4C-K1 在四级阈值下均明显低于 OPT4C-K16, 进一步验证多起点策略的必要性。

4.3 Python/NumPy CPU-LM 量级对照

为避免 GPU 加速结论脱离 CPU 求解语境, 本文补充一个单进程 Python/NumPy CPU-LM 基线。该基线复用与 CUDA runner 相同的 UR10 结构常量、raw target、Sobol seed、最大迭代次数和成功率阈值, 仅用于量级对照; 它不代表经过 C++、SIMD 或工业级工程优化的 CPU IK 实现。结果如表8所示, CPU-LM-K16 在 $N = 1000$ 时 Strict SR 与 OPT4C-K16 同为 0.954,

表 6 cuRobo-Graph 与 OPT4C 的系统级对比 ($N = 100, 500, 1000$)

N	方法	K	Strict SR	吞吐/ 10^4	全样本 $p95/mm$	成功 $p95/mm$	有效吞吐/ 10^4
100	OPT4C	16	0.960	1.502	4.38	3.17	1.442
100	cu-Graph	16	0.980	0.897	0.89	0.03	0.879
100	OPT4C	1	0.450	1.535	642.23	4.31	0.691
100	cu-Graph	1	0.860	1.038	74.17	0.05	0.893
500	OPT4C	16	0.954	1.734	4.34	2.10	1.654
500	cu-Graph	16	0.988	0.968	0.47	0.33	0.956
500	OPT4C	1	0.502	2.572	686.16	4.77	1.291
500	cu-Graph	1	0.838	4.350	114.46	0.22	3.645
1000	OPT4C	16	0.954	1.823	4.56	2.55	1.739
1000	cu-Graph	16	0.988	1.040	0.88	0.42	1.028
1000	OPT4C	1	0.522	2.786	685.01	4.61	1.454
1000	cu-Graph	1	0.840	7.262	88.67	0.28	6.100

注: cu-Graph 表示 cuRobo-Graph; 吞吐与有效吞吐单位均为 10^4 targets/s。

但吞吐仅 6.74 targets/s; OPT4C-K16 的对应吞吐为 1.72×10^4 – 1.82×10^4 targets/s。该对照说明当批量规模达到百级以上时, GPU target-block 并行结构能够显著摊薄固定调度和迭代开销。

表 7 不同误差阈值下的成功率扫描 ($N = 1000$)

方法	Loose	Medium	Strict	Ultra
OPT4C-K16	0.965	0.954	0.954	0.851
OPT4C-K1	0.568	0.540	0.522	0.249
cuRobo-Graph-K16	0.998	0.997	0.988	0.970
cuRobo-Graph-K1	0.896	0.857	0.838	0.825

表 8 Python/NumPy CPU-LM 量级对照

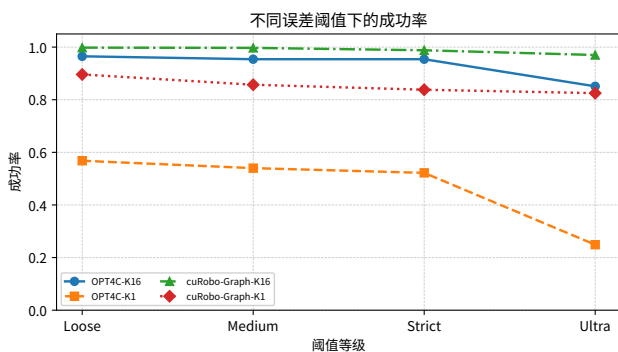
方法	N	Strict SR	吞吐/($t \cdot s^{-1}$)	$p95/mm$
CPU-LM-K1	100	0.450	94.20	642.2
CPU-LM-K16	100	0.960	6.33	4.38
CPU-LM-K1	500	0.502	102.97	686.2
CPU-LM-K16	500	0.954	6.66	4.34
CPU-LM-K1	1000	0.522	106.78	685.0
CPU-LM-K16	1000	0.954	6.74	4.56

4.4 种子数消融实验: 多起点策略的影响

为定量分离 Sobol 多起点策略对方法性能的贡献, 将种子数从默认的 $K = 16$ 降至 $K = 1$ (仅保留每组 Sobol 序列的第一个种子), 保持其他全部参数不变。 $K = 1$ 时 16 个线程中仅 lane 0 执行 LM 迭代, 其余 lane 空闲, 块内候选选择退化为直接输出。如表 9 所示, 消融结果揭示以下关键事实。

表 9 种子数消融实验: $K = 16$ vs $K = 1$

N	$K16$ SR	$K1$ SR	$K16$ $p95/mm$	$K1$ $p95/mm$
100	0.960	0.450	4.38	642.2
500	0.954	0.502	4.34	686.2
1000	0.954	0.522	4.56	685.0

图 8 不同误差阈值下的成功率扫描 ($N = 1000$)

进一步扫描 $K = 1, 2, 4, 8, 16$ 的结果如图 9 所示。随着 K 增大, Strict 成功率从 $K = 1$ 的 0.522 提升至 $K = 16$ 的 0.954, 而吞吐量从约 2.55×10^4 targets/s 降至约 1.72×10^4 targets/s。 $K = 8$ 到 $K = 16$ 仍带来 3.0 个百分点的成功率提升, 并将全样本 $p95$ 从 36.1

mm 降至 4.6 mm, 因此本文选择 $K = 16$ 作为默认成功率优先配置。

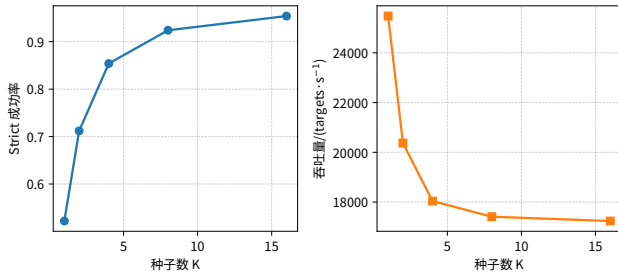


图 9 种子数扫描实验

消融结果揭示以下关键事实:

(1) **成功率主要来源于多起点策略。** $K = 16$ 时 Strict SR 稳定在 0.95 以上, $K = 1$ 时骤降至 0.45–0.52——降低约 43–51 个百分点。 $K = 1$ 的 p_{95} 误差为 642–685 mm ($K = 16$ 时为 4.3–4.6 mm), 表明单种子时大量求解陷入不可接受的局部极小值, 而 16 个 Sobol 低差异起点减少了大误差失败。

(2) **速度与成功率之间存在明确权衡。** 在补充扫描中, $N = 1000$ 时 $K = 1$ 吞吐量为 2.55×10^4 targets/s, $K = 16$ 为 1.72×10^4 targets/s; 以约三分之一的吞吐代价, Strict SR 从 0.522 提升至 0.954, 失败样本尾部误差收敛。这说明默认 $K = 16$ 是成功率优先配置, 而非纯吞吐最优配置。

(3) **核函数启动次数与 K 无关。** 无论 $K = 16$ 还是 $K = 1$, 核心求解阶段均由单个 target-block kernel 完成, 调度开销始终 $< 0.01\%$ GPU 流时间。这是该方法区别于多阶段 GPU pipeline 的主要结构差异。

(4) **与 cuRobo 的交叉对比确立范式边界。** 同等单种子条件下, cuRobo SR 为 0.830–0.880, 该方法仅为 0.450–0.522——cuRobo 的 L-BFGS 优化器局部收敛能力更强。同等 16 种子条件下 (如表 6 所示), cuRobo SR 达 0.988 但吞吐仅 1.04×10^4 , 该方法以 0.954 SR 实现 1.82×10^4 吞吐 (1.75 倍优势)。三方对比确立清晰的帕累托前沿: cuRobo $K=16$ 位于高精度端, 该方法 $K=16$ 位于吞吐优先且满足 Strict 阈值的区域, cuRobo $K=1$ 为吞吐优先且失败尾部较重。该方法的定位由此明确——不是通用 IK 优化器的替代品, 而是在满足工程精度阈值前提下, 通过单核函数融合实现更高吞吐的硬件-算法协同设计。

4.5 近奇异与近限位鲁棒性实验

为检验随机可达目标之外的适用边界, 构造 wrist、elbow、shoulder 三类近奇异目标和一类近限位目标, 每类覆盖 $N = 100, 500, 1000$ 。所有目标仍采用固定随机

种子 42 并由 FK 生成, 因而保持物理可达。近奇异数据按 q_1 – q_6 的物理关节编号描述: wrist singular 将 q_5 限制在 $[-0.03, 0.03]$ rad; elbow singular 将 q_3 设置为距下限或上限 0.05 rad; shoulder singular 将 q_1 限制在 $[-0.05, 0.05]$ rad, 并将 q_2 设置为距下限或上限 0.05 rad。对应脚本中的 0-based index 分别为 $q[4]$ 、 $q[2]$ 、 $q[0]$ 和 $q[1]$ 。近限位数据则先随机采样关节角, 再对每个目标随机选择一个关节、随机选择上下限方向, 并将其设置为距限位 0–0.087 rad。上述规则对应的 raw target 均由同一 UR10 FK 管线生成。

如图 10 所示, OPT4C-K16 在 wrist 和 shoulder 类型下仍保持较高成功率; 但 elbow singular 下 $N = 1000$ 的 Strict SR 降至 0.883, 明显低于随机可达目标的 0.954。这说明 Sobol 多起点能够缓解奇异附近的局部极小和条件数恶化, 但固定阻尼 LM 仍不能完全处理更困难的肘部奇异区域。

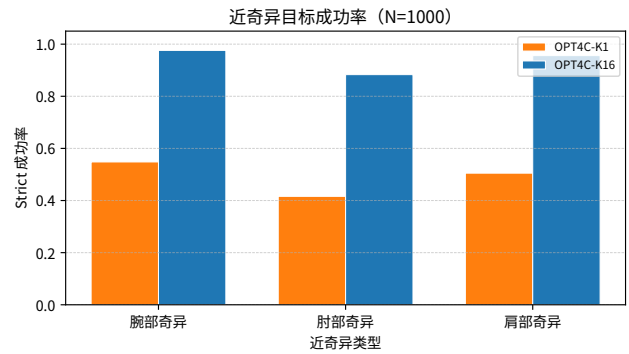


图 10 近奇异目标成功率 ($N = 1000$)

近限位实验使用至少一个关节距限位小于 0.087 rad 的 FK 目标, 并比较关节限位正则 ON/OFF。结果如图 11 所示, 在 $N = 1000, K = 16$ 下正则 ON 与 OFF 的 Strict SR 分别为 0.940 和 0.943, near-limit ratio 分别为 0.015 和 0.017。该结果表明当前 barrier 的主要作用不是显著提高 Strict SR, 而是在保持相近求解质量的同时轻微降低近限位解比例; 在该批 near-limit 目标上, $w_{\text{limit}} = 0.03$ 甚至使全样本 p_{95} 从 9.45 mm 增至 11.90 mm。因此论文将 barrier 定位为安全裕度约束, 而非主要收敛加速手段。

关节限位权重扫描进一步显示, 在 near-limit 目标上提高 w_{limit} 不会单调提升 Strict SR, 且过大权重可能扩大失败样本尾部误差。默认 $w_{\text{limit}} = 0.03$ 是保守折中, 而不是严格最优参数。相关趋势见图 12。

4.6 轨迹连续性实验

真实机器人通常沿连续轨迹求解 IK, 而非孤立目标。本文构造 line_50、arc_50 和 random_local_50 三

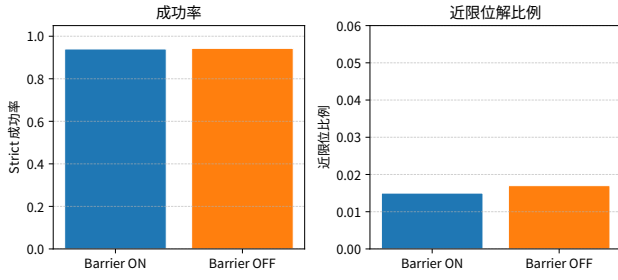


图 11 近限位目标下 Barrier ON/OFF 对比

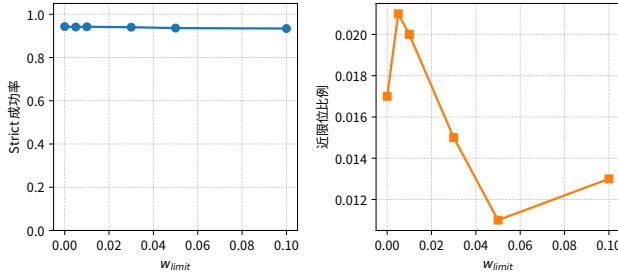


图 12 Barrier 权重扫描

类轨迹, 每类 20 条、每条 50 点。相邻点关节跳变量采用旋转关节环绕差分计算, 即 $\Delta q = \text{atan2}(\sin(q_t - q_{t-1}), \cos(q_t - q_{t-1}))$, 再取 6 维欧氏范数; joint jump 统计阈值为 0.5 rad。由于单点 pose cost 选择可能在相邻时刻切换 IK 分支, 原始 best 选择的 $p95(\Delta q)$ 为 5.44–5.61 rad, jump count 为 883–893。使用候选级离线 smoothness rerank 后, $p95(\Delta q)$ 降至 3.48–3.80 rad, jump count 降至 718–778, 点级成功率和轨迹级成功率保持不变, 如图 13 所示。由于每类轨迹包含 $20 \times 49 = 980$ 个相邻间隔, smoothness rerank 后 jump ratio 仍约为 73.3%–79.4%, 说明当前候选级重排序只能降低跳变幅值, 不能从根本上保证关节连续性。该结果说明简单二次平滑重排序不能替代基于上一时刻解的 warm-start、时间参数化或连续性约束优化; 本文轨迹实验仅用于评估 IK 分支连续性, 不构成控制器级轨迹安全性证明。

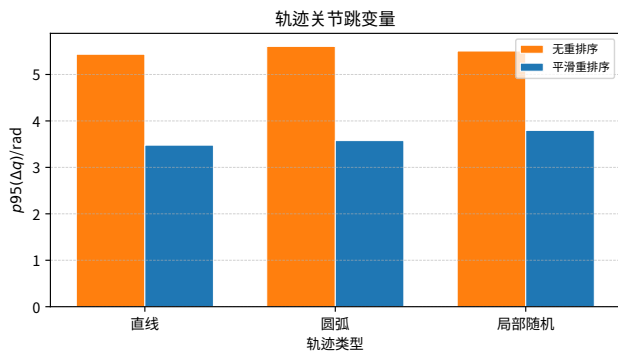


图 13 轨迹关节跳变量对比

4.7 最大迭代次数扫描

图 14 给出了 $N = 1000, K = 16$ 下的最大迭代次数扫描。 $\max_iter = 20$ 时 Strict SR 为 0.912, $p95$ 为 25.7 mm; 增至 60 后 Strict SR 达 0.954, $p95$ 降至 4.6 mm。继续增至 80 或 100 仅带来 0.3–0.5 个百分点的成功率增益, 却近似线性增加计算时间, 因此本文采用 $\max_iter = 60$ 作为默认折中。该扫描与主基准属于独立批次, 吞吐量只用于同图内相对趋势, 不反向覆盖表 5 的主结果。

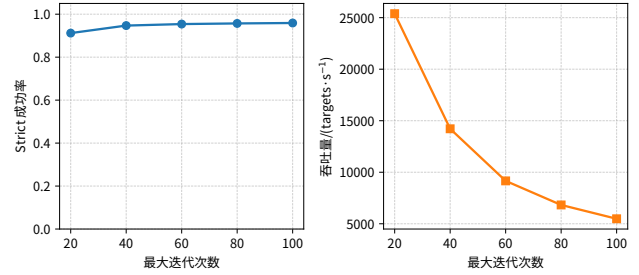


图 14 最大迭代次数扫描

4.8 微架构瓶颈的 PTX 级定量分析

为进一步探究吞吐量渐近收敛的微架构成因, 从 PTX 汇编层面进行定量分析。

寄存器压力与占用率。表 10 列出不同寄存器限制下主核函数的统计。无限制时每线程 194 个寄存器, 限制 160 时开始溢出 (216 bytes spill stores), 限制 128 时溢出显著增加 (568 bytes spill stores/740 bytes spill loads)。RTX 4060 Laptop GPU 每 SM 65 536 寄存器, 每线程束 $32 \text{ 线程} \times 194 \text{ 寄存器} = 6208 \text{ 寄存器}$, 每 SM 最多驻留 $65536/6208 \approx 10.5$ 线程束, 每块需 2 线程束 (32 线程), 故每 SM 最多 5.3 块, 理论占用率 $10.5/24 \approx 44\%$ 。

表 10 不同寄存器限制下的 PTX 寄存器与溢出统计

限制	寄存器/线程	溢出存储/B	溢出加载/B
无限制	194	0	0
160	160	216	212
128	128	568	740

44% 的中等占用率表明当前核函数的瓶颈不是单纯由全局内存带宽决定。结合 PTX 指令结构, 长延迟依赖主要来自 6×6 FP64 小矩阵求解、三角函数、寄存器相关依赖和标量控制流。进一步优化方向不应只放在数据搬运或 occupancy 提升, 而应优先降低单迭代小矩阵求解和超越函数的延迟; 下文通过混合精度消融实验对该判断进行定量检查。

共享内存访问分析。候选解存储采用 kCandidateStride=16 的列步长布局,总用量 2 064 bytes (< 5% of 48 KB/SM)。Nsight Compute 采样显示共享内存相关指标远小于 FP64 指令等待和寄存器相关 stall,因此该部分不是当前主瓶颈。本文不把共享内存访问作为主要贡献点。

浮点吞吐利用率估算。以 $N = 1000$ 为例:每目标约 20 次迭代 $\times$ 每迭代约 1 650 FLOP = 33 000 FLOP/target,总计 3.3×10^7 FLOP 在 54.9 ms 内完成,实际 FP64 吞吐约 0.60 GFLOPS——仅为 RTX 4060 Laptop GPU 理论峰值 (约 180 GFLOPS) 的 0.33%。极低的浮点利用率源于大量标量寄存器操作、超越函数 (sin/cos/atan2) 和控制流指令——小型 6×6 矩阵运算中传统 FLOP/s 指标不适用,这也是小矩阵 GPU 计算的普遍特征。

混合精度验证实验。上述 PTX 分析表明,仅将可向量化部分改为 FP32 未必能解除主要长延迟依赖。为验证这一判断,将雅可比构造与 Hessian 累加迁移至 FP32 (mixed_safe 模式),仅保留 6×6 高斯消元与收敛判定在 FP64。结果如表 11 所示: $N = 1000$ 时吞吐量仅从 1.89×10^4 提升至 1.93×10^4 targets/s (+2%),Strict SR 与 p_{95} 基本持平。该结果说明,在当前实现中,性能限制并非单独来自雅可比和 Hessian 的浮点精度选择,而与寄存器级小矩阵求解、超越函数和控制流共同相关。

表 11 混合精度消融实验 ($N=1000, K=16$)

配置	Strict SR	吞吐/(t · s ⁻¹)	p_{95} /mm
FP64 (基准)	0.954	18930	4.6
mixed_safe (FP32 J/H)	0.954	19275	4.5

注:表 11 数据来自独立实验批次,其 FP64 基线 (18930 targets/s) 与表 5 (18226 targets/s) 存在约 4% 的批次间差异,仅表内对比有效。

Nsight Compute 动态验证。为进一步检查 PTX 静态分析的判断,使用 NVIDIA Nsight Compute[24]对主核函数 ($N = 1000, K = 16$) 进行运行时采样,关键动态指标如表 12 所示。Warp Stall Long Scoreboard 占比为 83.2%,说明线程束长时间等待前序操作就绪;结合代码结构,其来源可能包括 FP64 小矩阵求解、三角函数、寄存器依赖和标量分支,而不能简单等同为单一 FP64 管线延迟。Issue Slot Utilization 为 2.32%,表明指令发射单元存在大量空闲周期。Memory Throughput 仅为 3.71%,共享内存 Bank 冲突计数约 112k 次,说明全局或共享内存带宽不是当前主要限制因素。

表 12 Nsight Compute 动态性能指标 ($N=1000, K=16$)

指标	实测值	含义
Warp Stall Long Scoreboard	83.2%	长延迟依赖占主导
Issue Slot Utilization	2.32%	指令发射利用率低
Compute (SM) Throughput	84.2%	活跃时计算密集
Memory Throughput	3.71%	非访存瓶颈
共享内存 Bank 冲突	~112k	冲突率 < 0.1%

4.9 批量扩展性分析

图 15 展示了吞吐量和 GPU 时间随 N 的变化趋势。吞吐量从 $N = 100$ 的 1.50×10^4 渐近收敛至 $N = 1000$ 的 1.82×10^4 targets/s,符合 Amdahl 定律预测——lane 0 串行候选选择构成不可并行化串行段。GPU 流时间严格线性 ($R^2 > 0.999$),每目标固定开销 ≈ 0.055 ms,用户无需对不同批量规模进行预扫描或参数调优即可精确预估吞吐量——这是定制 CUDA 线程映射相对于通用 GPU 调度器的核心优势。

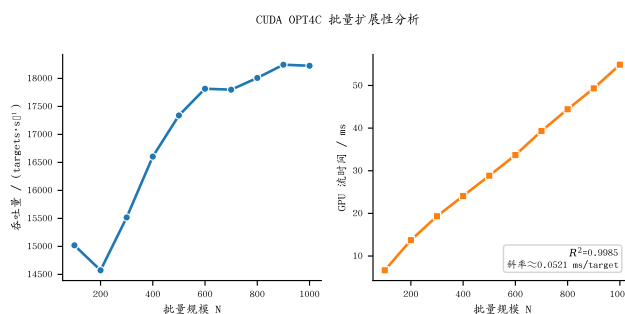


图 15 批量扩展性分析

5 讨论

5.1 设计范式:帕累托前沿上的定位

消融实验与公平对比共同确立了该方法在“质量-吞吐量”帕累托前沿上的位置。cuRobo $K=16$ 凭借 L-BFGS 的收敛能力位于高精度端 (SR 0.988, p_{95} 0.9 mm),但在本研究的 $N \leq 1000$ 设置下吞吐为 1.04×10^4 targets/s。cuRobo $K=1$ 位于吞吐优先端 (7.26×10^4 targets/s),但失败尾部较重,导致全样本 p_{95} 明显升高;其成功样本精度仍然较高。该方法 $K=16$ 位于两者之间:SR 0.954, p_{95} 4.6 mm、吞吐 1.82×10^4 targets/s。

这一拐点定位的工程意义明确:在采样式运动规划的典型场景 ($N \leq 1000$, 单次查询需一次性高成功率),该方法提供了无需后处理、延迟确定的求解路径。4.6 mm 的 p_{95} 虽高于 cuRobo $K=16$ 的 0.9 mm,但满足本文 Strict 阈值;在中等精度前端任务中,1.75 倍

的吞吐增益具有实际价值。若应用场景要求亚毫米级精度（如精密装配），cuRobo $K=16$ 更为合适，这也划定了该方法的适用边界。

该方法的主要贡献由此明确为**小矩阵批量计算中硬件-算法协同设计带来的结构差异**，而非单一优化器的性能超越。

5.2 可推广性与加速方向

本文框架可推广到其他串联机械臂，但需要重新生成结构参数、解析雅可比计算路径、线程映射和寄存器资源配置。对于 7-DOF 冗余机械臂（如 Franka Panda[25]），解空间维度增加且零空间优化需求更强，不能直接认为 UR10 上的成功率和吞吐量可平移。两个后续方向更值得优先研究：（1）利用相邻目标或上一时刻解进行 seed warm-start，减少平均迭代次数并改善轨迹连续性；（2）引入奇异鲁棒阻尼或任务相关重排序，缓解 elbow singular 和 near-limit 目标中的失败尾部。

6 结论

针对固定 6-DOF 中小批量 IK 中单种子收敛概率不足与多阶段 GPU pipeline 固定开销较高的问题，本研究提出结构感知单核融合 CUDA 实现。主要结论如下：

（1）**Sobol 多起点 + Kernel Fusion 协同设计**：将 $K = 16$ Sobol 种子、解析雅可比、LM 迭代和块内候选选择融合进单个 target-block kernel。种子数扫描表明， K 从 1 增至 16 时 Strict SR 从 0.522 提升至 0.954，全样本 $p95$ 从 685.0 mm 降至 4.6 mm。

（2）**融合 FK + 解析雅可比**：单次 FK 输出 T_{ee} , p_i , z_i ，雅可比由 6 次叉积组装，每迭代 FK 调用从有限差分所需的多次扰动评估降至 1 次结构化评估，减少了重复 FK 调用并提高了 LM 方向构造的一致性。

（3）**低控制流复杂度 LM + 寄存器级 6×6 求解**：摒弃 acceptance-rejection 回滚分支，降低控制流复杂度；高斯消元主要在寄存器内完成。

（4）**公平对比确立帕累托前沿定位**：同等 16 种子下，cuRobo SR 0.988 位于高精度端，该方法 SR 0.954 满足 Strict 阈值且吞吐为其 1.75 倍。该优势主要来自单核函数融合减少中间同步与固定结构 LM 较轻量计算路径的共同作用，而不是单一地来自 kernel launch 或调度开销。三方对比（cuRobo $K=16$ / 该方法 $K=16$ / cuRobo $K=1$ ）确立清晰的帕累托前沿。

（5）**鲁棒性和边界**：近奇异实验显示 elbow singular 下成功率下降至 0.883；近限位实验显示 barrier 主

要作为安全裕度约束而非成功率来源；轨迹实验显示 smoothness rerank 能降低 $p95(\Delta q)$ ，但 jump ratio 仍然较高，不能充分抑制分支跳变。

综上，在 $N \leq 1000$ 、固定 6-DOF、无碰撞约束、可达目标主要由 FK 生成的条件下，该方法提供了一种成功率优先、执行路径确定的 CUDA IK 实现方案。其优势来自 Sobol 多起点与单 kernel fusion 的协同，而非单个 LM 优化器的局部收敛能力。本文方法当前不处理碰撞约束，不保证连续控制轨迹，不面向亚毫米级精密装配任务；其主要适用场景是固定 6-DOF 机械臂在中小批量目标下的规划前端 IK 候选生成。对于更大批量、更高精度、复杂碰撞约束或亚毫米装配场景，cuRobo 等通用 GPU 规划框架仍具有重要优势。

后续工作包括：结合 warm-start 与连续性约束降低轨迹跳变；引入奇异鲁棒阻尼；在真实 UR calibration 模型和 7-DOF 冗余机械臂上重新评估成功率、吞吐量和寄存器资源。

参考文献

- [1] Siciliano B, Sciavicco L, Villani L, et al. Robotics: modelling, planning and control[M]. Springer, 2010.
- [2] Lynch K M, Park F C. Modern robotics: mechanics, planning, and control[M]. Cambridge: Cambridge University Press, 2017.
- [3] LaValle S M. Planning algorithms[M]. Cambridge University Press, 2006.
- [4] Sucan I A, Moll M, Kavraki L E. The Open Motion Planning Library[J]. IEEE Robotics & Automation Magazine, 2012, 19(4): 72-82.
- [5] Thomason W, Kingston Z, Kavraki L E. Motions in microseconds via vectorized sampling-based planning[C]//2024 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2024: 8749-8756.
- [6] NVIDIA. CUDA C++ programming guide[EB/OL]. [2026-06-10]. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [7] Nickolls J, Buck I, Garland M, et al. Scalable parallel programming with CUDA[J]. Queue, 2008, 6(2): 40-53.

- [8] Sundaralingam B, et al. cuRobo: parallelized collision-free robot motion generation[EB/OL]. arXiv: 2310.17274, 2023.
- [9] Hsiao Y S, Hari S K S, Sundaralingam B, et al. VaPr: variable-precision tensors to accelerate robot motion planning[C]//2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2023.
- [10] Yasutake C, Kingston Z, Plancher B. HJCD-IK: GPU-accelerated inverse kinematics through batched hybrid Jacobian coordinate descent[EB/OL]. arXiv: 2510.07514, 2025.
- [11] Abuelsamen L, Rana H, Lu H W, et al. Industrial robot motion planning with GPUs: integration of cuRobo for extended DOF systems[EB/OL]. arXiv: 2508.04146, 2025.
- [12] Whitney D E. Resolved motion rate control of manipulators and human prostheses[J]. IEEE Transactions on Man-Machine Systems, 1969, 10(2): 47-53.
- [13] Wampler C W. Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods[J]. IEEE Transactions on Systems, Man, and Cybernetics, 1986, 16(1): 93-101.
- [14] Chan S K, Lawrence P D. General inverse kinematics with the error damped pseudoinverse[C]//Proceedings. 1988 IEEE International Conference on Robotics and Automation. IEEE, 1988: 834-839.
- [15] Nakamura Y, Hanafusa H. Inverse kinematic solutions with singularity robustness for robot manipulator control[J]. Journal of Dynamic Systems, Measurement, and Control, 1986, 108(3): 163-171.
- [16] Chiaverini S. Singularity-robust task-priority redundancy resolution for real-time kinematic control of robot manipulators[J]. IEEE Transactions on Robotics and Automation, 1997, 13(3): 398-410.
- [17] Universal Robots. Universal Robots ROS2 description[EB/OL]. [2026-06-10]. https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.
- [18] Buss S R. Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods[J]. IEEE Transactions on Robotics, 2004, 20(1): 1-18.
- [19] Levenberg K. A method for the solution of certain non-linear problems in least squares[J]. Quarterly of Applied Mathematics, 1944, 2: 164-168.
- [20] Marquardt D W. An algorithm for least-squares estimation of nonlinear parameters[J]. Journal of the Society for Industrial and Applied Mathematics, 1963, 11(2): 431-441.
- [21] Sobol I M. On the distribution of points in a cube and the approximate evaluation of integrals[J]. USSR Computational Mathematics and Mathematical Physics, 1967, 7(4): 86-112.
- [22] Abdelfattah A, Haidar A, Tomov S, et al. Batched one-sided factorizations of tiny matrices using GPUs: challenges and countermeasures[J]. Journal of Computational Science, 2018, 26: 226-236.
- [23] NVIDIA. CUDA C++ best practices guide[EB/OL]. [2026-06-10]. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>.
- [24] NVIDIA. Nsight Compute user guide[EB/OL]. [2026-06-10]. <https://docs.nvidia.com/nsight-compute/>.
- [25] Chitta S, Sucas I, Cousins S. MoveIt![J]. IEEE Robotics & Automation Magazine, 2012, 19(1): 18-19.

作者简介

刘霄鹏 (2005—), 男, 本科, 主要研究方向为机器人运动学、GPU 并行计算。

E-mail: 182625516@qq.com